

1. Identidad de la aplicación

- ¿Qué nombre le pondrías a tu aplicación?
- ¿En qué temática o rubro se basará (educación, salud, finanzas, entretenimiento, etc.)?
- En una frase corta, ¿cómo describirías la idea principal de tu aplicación?
- NOMBRE APP: LabHelper
- Está orientada al rubro de gestión de laboratorio, particularmente laboratorios químicos, ambientales, industriales, de investigación o control de calidad.
- Sistema de gestión de laboratorio para registrar, organizar y dar seguimiento al ciclo completo de análisis de muestras, ensayos y resultados.
- Organizar las muestras , ensayos y resultados de los experimentos y/o servicios.

2. Usuarios y perfiles

- ¿Quiénes serían los usuarios principales de la aplicación?
- ¿Qué roles o perfiles diferentes existirían (ej: administrador, usuario estándar, invitado)?
- ¿Qué diferencia habría entre lo que puede hacer cada perfil?
- Los usuarios principales: analistas o técnicos de laboratorio.
- Roles:
 - Analistas/tecnicos de laboratorio : responsable de registrar muestras, ensayos y cargar los resultados.
 - Supervisor/Coordinador: revisa, verifica y aprueba resultados antes de emitir un informe de los mismos.
 - Administrador del sistema: configura el sistema.

Administrador	Analista	Supervisor
• crear usuarios • editar usuarios	• registrar muestras • consultar muestras	• visualizar resultados cargados

• asignar roles • gestionar tipos de ensayo • gestionar equipos • consultar toda la información	• asignar ensayos • cargar resultados experimentales • modificar resultados antes de aprobación	• aprobar resultados • rechazar resultados • solicitar repetición del análisis • cerrar muestras finalizadas
--	---	---

3. Funcionalidades principales

- ¿Cuáles son las 3 funcionalidades más importantes que sí o sí querés que tenga la aplicación?
- ¿Qué funcionalidades considerarás “deseables pero no indispensables”?
- ¿Qué **NO** debería hacer tu aplicación (cosas que están fuera de alcance)?
- Las 3 funcionalidades más importantes que debe tener:
 - Registrar y gestionar las muestras: con los atributos de código, tipo, cliente , fecha de ingreso, observaciones, estado, fecha de salida.
 - Gestionar ensayos y cargar resultados: asociar ensayos (pH, conductividad, viscosidad, electroquímica,etc) a una muestra y cargar resultados.
 - Seguimiento y trazabilidad del proceso completo: visualizar el estado de cada muestra: RECIBIDO, EN_ANALISIS, REVISION_PENDIENTE, APROBADO, REPORTADO, REPETIR

¿Qué funcionalidades serían deseables pero no indispensables?

Informes de resultados en PDF, dashboard, búsqueda por filtros, notificaciones, carga masiva de datos CSV, historial tipo registro de auditoria de muestras.

¿Qué **NO** debería hacer la aplicación?

Conexiones con equipos o instrumentos de laboratorio, facturación o operaciones comerciales.

4. Requerimientos básicos

- ¿Necesitará autenticación (usuario/contraseña, login)?
- ¿Qué información mínima debería manejar tu aplicación (ej: usuarios, productos, cursos, reservas)?

¿Necesitará autenticación?

Sí. Mas adelante se implementara autenticación JWT con Spring Security.

¿Qué información mínima debería manejar?

Usuarios(nombre, usuario, contraseña, rol)

Muestras(código, tipo, cliente, fecha de ingreso, observaciones, estado, fecha de salida)

Cuando sean muestras propias, se nombrara en el atributo cliente "Uso Interno" o "Propio".

Si quisiera escalar la app, podria pensar en clases hijas de muestras (MuestraInterna con atributo proyectoAsociado y MuestraComercial con atributo cliente y costoServicio)

Ensayos(nombre, tipo de ensayo, descripción, fecha asignación, responsable)

Resultados(valores obtenidos, unidad, fecha, observaciones)

Equipos(nombre equipo, estado, fecha de calibración, mantenimiento)--VER

5. Casos de uso / circuitos

- Describí **3 circuitos completos de principio a fin** que un usuario pueda realizar dentro de tu aplicación.

Ejemplo:

1. Crear una cuenta nueva → recibir confirmación → poder iniciar sesión.
2. Cargar un producto/servicio → guardarlo en base de datos → mostrarlo en una lista.
3. Solicitar una acción (ej: reservar turno) → confirmarla → consultar historial de reservas.

Circuito 1 — Registro de muestra

Usuario inicia sesión--> registra nueva muestra-->sistema genera código interno --> la muestra se guarda en el sistema--> aparece disponible para asignar ensayo

Circuito 2 — Carga de resultados

Analista selecciona una muestra-->verifica que este asignado el ensayo correspondiente, si no se lo asigna--> carga resultado de ensayo de la muestra--> sistema actualiza el estado en EN_ANALISIS.

Circuito 3 — Revisión y cierre

Supervisor visualiza resultados cargados por el analista--> revisa los mismos--> aprueba o rechaza--> si aprueba , la muestra cambia a APROBADO--> Si rechaza, la muestra cambia a REPETIR--> Se reporta en informe cambia a REPORTADO --> se cierra el análisis.

6. Expectativas y alcance temporal

- En 3 meses de desarrollo, ¿qué te gustaría tener sí o sí terminado y funcionando?
- ¿Qué cosas podrías dejar para una “versión 2” futura?

Me gustaría tener funcionando:

- Ingreso por usuario y contraseña.
- Visualizar las muestras disponibles y registrar nuevas.
- Buscar por código, por tipo de ensayo, por responsable, por cliente a futuro.
- Cargar resultados, registro de todo el ciclo completo, emitir informe parciales y finales.
- Persistencia en base de datos.

Para una segunda versión futura incorporaría:

- API REST con Spring Boot
- autenticación JWT con Spring Security
- interfaz web con React
- reportes PDF
- dashboard de métricas
- auditoría completa
- gestión de equipos
- calendario de calibraciones
- despliegue con Docker

7. Inspiración

- ¿Conocés alguna aplicación o sistema similar al que querés hacer? ¿Qué te gusta de ella y qué mejorarías?

8. Comentarios

Tené en cuenta que, al desarrollar una aplicación completa (persistencia–modelo–vista) en un período de 3 a 4 meses, habrá ciertas tecnologías o funcionalidades que quedarán fuera del alcance debido a su complejidad o a los recursos que requieren. Algunos ejemplos pueden ser: el envío automático de correos electrónicos, integraciones con redes sociales o el uso de APIs de inteligencia artificial, entre otras.

7. La inspiración principal proviene de los sistemas LIMS (Laboratory Information Management System) utilizados en laboratorios de análisis químico, ambiental e investigación.

Lo que resulta valioso de estos sistemas es: orden, trazabilidad, historial de resultados, centralización de datos.

Lo que se busca mejorar con LabHelper es: interfaz más simple de usar, menor complejidad para labo pequeños o adaptado para estudiantes de grado o doctorado, facilidad de uso, posibilidad de escalar

8.

La primera versión se desarrollará como aplicación de Spring con Java con el objetivo de enfocarse en:

- modelado orientado a objetos
- lógica de negocio
- estructura del proyecto
- persistencia de datos

Luego evolucionará hacia una API REST, JWT con Spring Security, capa de vista con React.